

Server Manage 운영 위키

DECS 서버 관리 코드의 기능, 경계와 설계 의도

문서 기준: 2026-07-17 저장소 상태

소스: /home/jy/server_manage/wiki

목차

1. 전체 구조
2. container-images
3. kerberos-nfs
4. monitoring
5. remote-operations
6. server-state
7. user-lifecycle

System

원본: md/system/index.md

개요

System 영역은 **DECS GPU 서버와 사용자 container를 일관된 상태로 운영하기 위한 시스템 관리 코드**를 담당한다. 새로운 서버를 기존 서버와 같은 설정으로 구축하고, 사용자가 사용할 container와 image를 관리하며, 서버 전원부터 monitoring과 공유 storage 접근까지 운영에 필요한 전체 흐름을 유지하는 것이 목적이다.

주요 담당 범위는 다음과 같다.

- 모든 GPU 서버의 Docker, NVIDIA, Kubernetes와 network 공통 설정을 점검하고 신규 서버에도 같은 기준을 적용한다.
- 사용자 container의 생성·수정·삭제와 공통 container image를 관리한다.
- 서버 원격 부팅과 부팅 순서를 관리하여 사용자 관리 DB와 GPU 서버가 올바른 순서로 준비되도록 한다.
- 서버, GPU, container와 사용자 상태를 수집하여 장애와 이상 상태를 관측한다.
- AD/Kerberos 인증을 사용해 사용자가 NAS/NFS 공유 storage에 안전하게 접근하도록 한다.

즉, Backend나 Frontend 애플리케이션 기능이 아니라 **GPU 서버에서 사용자 작업 환경이 실제로 생성되고, 실행되고, 관측되고, storage에 연결되는 운영 기반**을 관리하는 영역이다.

목차

구성요소	상세 문서	PDF
전체 통합 매뉴얼	-	PDF 열기
전체 구조	현재 페이지	PDF 열기
container-images/	문서 열기	PDF 열기
user-lifecycle/	문서 열기	PDF 열기
server-state/	문서 열기	PDF 열기
remote-operations/	문서 열기	PDF 열기
monitoring/	문서 열기	PDF 열기
kerberos-nfs/	문서 열기	PDF 열기

container-images 설계·운영 매뉴얼

원본: md/system/container-images.md

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

1. 책임 범위

`container-images` 는 여러 CUDA/TensorFlow 조합을 동일한 사용자 경험으로 제공한다. 이미지가 책임지는 것은 OS·CUDA·Python/Jupyter 패키지와 컨테이너 안의 계정, SSH, 선택적 VNC, Kerberos ccache 사용 환경이다.

이미지는 UID/GID를 결정하지 않는다. 사용자 identity, 포트, 홈 디렉터리와 Kerberos keytab은 `user-lifecycle` 와 호스트가 관리한다. 이 경계를 지켜야 이미지가 사용자 DB나 특정 서버에 종속되지 않는다.

2. 디렉터리 지도

경로	핵심 기능
<code>Dockerfile</code>	모든 variant가 공유하는 단일 이미지 정의
<code>image-variants.json</code>	base image, CUDA/TensorFlow, 최소 driver, alias의 권위 있는 목록
<code>entrypoint.sh</code>	실행 시 사용자·그룹·SSH·Jupyter·VNC·Kerberos 환경 구성
<code>scripts/variant_matrix.py</code>	manifest를 Docker/GitHub Actions build matrix로 변환
<code>scripts/build_variants.py</code>	로컬 build/push 명령 생성과 실행
<code>scripts/test_image_variants.py</code>	이미지 메타데이터, TensorFlow와 선택적 GPU smoke test
<code>scripts/test_uid_create_container.py</code>	<code>user-lifecycle</code> CLI와의 dry-run 계약 검증
<code>tests/</code>	root_squash, Kerberos, sudo와 build 설정 회귀 테스트
<code>.github/workflows/docker-publish.yml</code>	PR merge 또는 수동 실행 시 matrix build/push

3. 핵심 기능

3.1 manifest 기반 이미지 variant

`image-variants.json` 한 곳에 다음 항목을 둔다.

- variant ID와 base NVIDIA CUDA image
- CUDA, TensorFlow, Python, Ubuntu 버전

- TensorFlow 설치 패키지와 conda 패키지 제약
- 필요한 최소 NVIDIA driver
- `stable` 또는 `experimental` 지원 상태
- 날씨가 없는 사용자용 alias

현재 안정 계열은 CUDA 11.8, 12.2, 12.3, 12.5이며 CUDA 12.8/H200 계열은 실장비 검증 전까지 experimental이다. 빌드 도구와 GitHub Actions가 같은 manifest를 읽으므로 로컬과 CI의 variant가 달라지지 않는다.

3.2 단일 Dockerfile

Dockerfile은 `BASE_IMAGE`, `CUDA_VERSION`, `TENSORFLOW_VERSION`, `MIN_NVIDIA_DRIVER`, `CONDA_PACKAGES` 등을 build argument로 받는다. 모든 variant는 공통으로 SSH, Kerberos client, 한국어 글꼴/입력기, Chrome, Xfce, TigerVNC/noVNC, Miniforge, Jupyter를 포함한다.

variant별 Dockerfile을 복제하지 않은 이유는 보안 패치나 공통 패키지 변경을 한 번만 적용하고, 차이는 manifest의 데이터로 검토하기 위해서다. 버전별 예외가 필요하면 먼저 manifest 값으로 표현하고, 정말 다른 설치 흐름일 때만 Dockerfile에 조건을 추가한다.

3.3 실행 시 identity 구성

`entrypoint.sh`의 주요 흐름은 다음과 같다.

1. 이미지 variant와 실제 host driver 정보를 출력한다.
2. `USER_ID`, `UID`, `GID`, `USER_GROUP`을 검증하고 기존 홈의 owner와 맞춘다.
3. supplemental group과 sudo 정책을 구성한다.
4. SSH 로그인과 Jupyter 설정을 만든다.
5. Kerberos 모드이면 전달받은 ccache를 사용자 환경에 연결한다.
6. Jupyter를 시작하고, 요청된 경우에만 VNC/noVNC를 시작한다.

호스트 mount가 `root_squash`인 환경에서는 root가 사용자 홈을 임의로 `chown` 할 수 없다. 그래서 이미 결정된 UID/GID로 사용자를 실행하고, 홈에 쓸 수 없는 root 작업을 최소화한다. 사용자 홈의 기존 비밀번호와 VNC password 파일도 재시작 때 보존한다.

3.4 제한적 sudo와 공유 helper

기본 `DECS_USER_SUDO_MODE=restricted`는 패키지 설치에 필요한 제한된 sudo는 허용하지만 UID 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행을 막는다.

사용자 그룹 공유는 관리자 sudo 대신 `group-dir-share` helper로 제공한다. 사용자 자신의 권한으로 디렉터리를 만들고 `2770`과 default ACL을 설정하므로 공유 기능과 identity 격리를 함께 유지한다.

Kerberos keytab과 ccache를 분리한 이유와 container credential 경계는 Kerberos/NFS의 keytab과 ccache 모델을 따른다.

3.5 GPU 호환성 표시와 강제 모드

이미지는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true` 일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

4. 빌드와 배포 흐름

```
image-variants.json
|
+--> variant_matrix.py --> GitHub Actions matrix --> build/push
|
+--> build_variants.py -----> local build/push
|
+--> test_image_variants.py -----> smoke test
```

전체 명령 확인:

```
cd /home/jy/server_manage/container-images
python3 scripts/build_variants.py --dry-run
```

특정 variant build:

```
python3 scripts/build_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

registry push는 build 결과와 tag 목록을 확인한 뒤 `--push` 를 추가한다. 날짜 tag는 재현 가능한 배포 지점을 만들고, `stable` / `latest` alias는 현재 권장 variant를 가리킨다. 운영 기록에는 alias보다 날짜가 포함된 tag를 남기는 것이 좋다.

5. 사용자 생명주기와의 계약

`uidctl create-container` 는 image 이름과 version tag, 최종 UID/GID, 포트와 home mount를 Docker 환경변수/ 옵션으로 전달한다. 이미지가 기대하는 주요 값은 다음과 같다.

값	의미
<code>USER_ID</code>	컨테이너 사용자 이름
<code>UID</code> , <code>GID</code>	DB·AD·NFS와 이미 일치가 검증된 숫자 identity
<code>USER_GROUP</code>	primary group 이름

값	의미
ENABLE_VNC	VNC/noVNC opt-in
KRB5CCNAME	컨테이너에 보이는 ccache 경로
DECS_KERBEROS_HOST_KEYTAB	호스트 관리 ticket을 기다리는 모드
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, NFS와 DB가 어긋나는 오류를 조기에 발견할 수 있다.

6. 테스트 전략

빠른 정적/회귀 테스트:

```
cd /home/jy/server_manage/container-images
bash tests/test_image_build_config.sh
bash tests/test_entrypoint_root_squash.sh
```

이미지 smoke test:

```
python3 scripts/test_image_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

실제 GPU까지 확인할 때만 --gpu 를 사용한다. 사용자 생성 계약은 실제 DB를 바꾸지 않는 dry-run/print 경로로 검증한다.

```
python3 scripts/test_uid_create_container.py \
  --variant cuda12.5-tf2.20-ubuntu22.04 \
  --print-only
```

7. 변경 가이드

새 variant 추가

1. image-variants.json 에 버전, base image와 최소 driver를 추가한다.
2. python3 scripts/variant_matrix.py 결과의 tag를 확인한다.
3. build dry-run과 shell 회귀 테스트를 실행한다.
4. 실제 image smoke test와 가능하면 대상 GPU host test를 수행한다.

5. 검증 전에는 `support: experimental` 과 명시적인 alias를 사용한다.

공통 런타임 변경

`Dockerfile` 변경과 `entrypoint.sh` 변경을 구분한다. package/파일은 build 시점에 넣고, 사용자 UID나 mount처럼 컨테이너마다 달라지는 항목만 `entrypoint` 에서 처리한다. `entrypoint` 변경은 재시작 시 idempotent한지, 기존 홈의 파일을 덮어쓰지 않는지, `root_squash` 환경에서 동작하는지를 함께 확인한다.

8. 운영 안전 수칙

- 실제 사용자 비밀번호를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.
- `latest` 만 기록하지 말고 장애 분석이 가능한 날짜 tag를 남긴다.
- `experimental variant`를 `stable alias`로 바꾸기 전에 실장비 GPU test를 한다.
- `sudo` 완화는 `ccache`와 `host bind mount`에 대한 우회 가능성을 먼저 검토한다.
- 로컬 source 변경과 이미 registry에 push된 image를 구분해서 진단한다.

Kerberos/NFS

원본: md/system/kerberos-nfs/index.md, md/system/kerberos-nfs/design.md, md/system/kerberos-nfs/operations.md

역할: FARM/LAB에서 AD Kerberos로 사용자를 인증하고, 같은 UID/GID로 NFS 공유 스토리지에 접근하게 하는 구조와 운영 기준을 설명한다.

이 문서는 Kerberos/NFS 문서의 시작점이다. 처음 읽는 사람은 **설계**에서 전체 인증 흐름을 먼저 이해한 뒤 **운영**에서 생성·점검·복구 명령을 확인한다. 환경별 실제 값과 최종 실행 절차는 인프라 저장소의 FARM/LAB canonical runbook을 단일 기준으로 삼는다.

문서 구성

문서	읽어야 하는 경우	핵심 내용
현재 페이지	전체 범위와 문서 위치를 빠르게 확인할 때	목적, 원칙, 소유 경계
설계	인증이 어느 서버와 프로세스를 거치는지 이해할 때	keytab 발급, ticket 발급, RPCSEC_GSS, NAS 권한 판정, 설정 근거
운영	계정/컨테이너 생성, timer 확인, mount-KVNO 장애를 처리할 때	명령, 점검표, 모니터링 코드, 수동 repair/rotation

한눈에 보는 구조

관리 서버(uidctl)

- └─ AD DC: 사용자·RFC2307·keytab 생성
- └─ 계산 호스트: root-only keytab + ccache refresh timer
- └─ NAS/storage: NFS service keytab + 사용자 home
- └─ container: keytab이 아닌 ccache만 사용

사용자 I/O

container process -> host kernel NFS client -> rpc.gssd
-> AD KDC ticket -> NAS svcgssd/NFS -> UID/GID 권한 판정

현재 구현의 중요한 경계는 다음과 같다.

- 사용자 keytab은 장기 자격증명이므로 계산 호스트의 `/etc/decs-krb/keytabs/<username>.keytab`에 `root:root0400`으로만 둔다.

- 컨테이너에는 keytab을 넣지 않는다. 호스트가 만든 `/run/user/<uid>/krb5cc` 와 읽기 전용 `/etc/krb5.conf` 만 bind mount한다.
- 컨테이너가 시작되기 전에 사용자별 `decs-krb-refresh@<username>.timer` 를 활성화하고 유효한 ccache를 한 번 발급한다.
- NFS mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 쓴다. FARM의 source는 `nas.farm.decs.internal:/volume1/share` 다.
- `addr=100.100.100.120` 은 FQDN이 해석된 실제 전송 주소로 사용할 수 있다. source 자체를 `100.100.100.120:/...` 로 바꾸면 안 된다.
- 기본 security flavor는 `sec=krb5` 다. `krb5i` 와 `krb5p` 는 무결성·암호화가 명시적으로 필요한 경로에서 성능 비용과 함께 선택한다.

Identity 불변 조건

한 사용자에게 대해 다음 숫자가 같아야 한다.

```
UID DB ubuntu_uid
= AD RFC2307 uidNumber
= NFS client에서 관측한 home owner UID
= container UID
```

primary GID도 같은 원칙 적용

이 조건을 확인하지 않고 NAS에서 `chown` 만 반복하면 AD, DB, NAS의 identity가 더 어긋날 수 있다. 계정 생성 흐름은 Docker와 DB를 확정하기 전에 AD/NAS/host identity와 실제 Kerberos NFS 쓰기를 검증한다.

모듈 소유 경계

작업	소유 모듈
AD 사용자·그룹, 사용자 keytab, host ccache timer, container 생성	<code>user-lifecycle</code>
공통 host Kerberos/NFS desired state와 fstab	<code>server-state</code>
NAS NFS service account·SPN·service keytab의 수동 변경	<code>kerberos-nfs</code>
GSS readiness, keytab drift 관측, mount 상태, canary, forensics	<code>monitoring</code>
ccache 소비, 시작 전 credential 확인, restricted sudo	<code>container-images</code>

관측 실패가 곧바로 AD password 변경이나 NAS service 재시작으로 이어지지 않게 읽기 전용 monitoring과 상태 변경 작업을 분리한다.

단일 기준 문서와 코드

- [FARM canonical runbook](#)
- [LAB canonical runbook](#)
- [user-lifecycle Kerberos 구현](#)
- [container 생성 구현](#)
- [host Kerberos/NFS desired state](#)
- [Kerberos/NFS keytab health check](#)
- [NFS GSS monitoring](#)

실제 endpoint, principal, mount option이 바뀌면 이 위키보다 canonical runbook과 코드 설정을 먼저 갱신하고, 검증 시각과 대상 host를 함께 기록한다.

Kerberos/NFS 설계

이 문서는 keytab이 만들어지는 시점부터 사용자가 NFS 파일을 읽고 쓰는 시점까지, 어느 서버의 어떤 객체와 프로세스가 관여하는지를 설명한다.

1. 설계 목표

- 비밀번호를 컨테이너에 저장하지 않고 사용자별 Kerberos 인증을 제공한다.
- DB, AD, 계산 호스트, NAS와 컨테이너가 같은 numeric UID/GID를 사용한다.
- NFS 서버는 FQDN 기반 service principal로 인증하고 client는 `sec=krb5` 로 RPCSEC_GSS context를 만든다.
- keytab 같은 장기 자격증명의 노출 범위를 host root로 제한한다.
- ticket 갱신과 컨테이너 lifecycle을 분리하여 컨테이너 재시작 중에도 credential을 안정적으로 유지한다.
- monitoring은 drift를 관측하되 공유 NAS와 AD를 자동으로 변경하지 않는다.

2. 전체 구성

flowchart LR

```
M["관리 서버<br/>uidctl + Ansible"]
AD["Samba AD DC / KDC<br/>사용자·그룹·SPN<br/>RFC2307·KVNO"]
H["FARM/LAB 계산 호스트<br/>user keytab<br/>refresh timer<br/>rpc.gssd + kernel NFS"]
C["사용자 container<br/>process + ccache<br/>KRB5CCNAME"]
NAS["NAS / storage<br/>svcgssd + NFS server<br/>service keytab<br/>winbind/idmap"]
MON["monitoring<br/>readiness·KVNO·mount<br/>canary·forensics"]

M -->|"samba-tool / exportkeytab"| AD
M -->|"root-only keytab 설치"| H
M -->|"home과 numeric owner 준비"| NAS
AD <-->|"AS/TGS, directory lookup"| H
H -->|"ccache bind mount"| C
C -->|"파일 I/O"| H
H <-->|"NFSv4 + RPCSEC_GSS"| NAS
NAS -->|"RFC2307 UID/GID 조회"| AD
MON -. ->|"읽기 전용 상태 수집"| AD
MON -. -> H
MON -. -> NAS
```

구성요소별 책임

위치	객체/프로세스	하는 일
관리 서버	uidctl, Ansible runner	계정 생성 transaction을 조정하고 keytab을 필요한 host로 전달
AD DC	사용자·그룹 객체, krbtgt, KDC, samba-tool	principal과 RFC2307 속성 저장, keytab export, TGT/service ticket 발급
계산 호스트	사용자 keytab	ticket을 새로 발급할 수 있는 장기 자격증명; root만 읽음
계산 호스트	decs-krb-refresh@.service/.timer	keytab으로 ccache를 만들고 갱신하여 원자적으로 교체
계산 호스트	kernel NFS client, rpc.gssd	호출 UID의 credential로 NFS RPCSEC_GSS context 생성
컨테이너	사용자 process, bind-mounted ccache	KRB5CCNAME 이 가리키는 ticket을 사용하고 NFS 파일 I/O 수행
NAS/storage	svcgssd, NFS server, service keytab	nfs/<fqdn>@<realm> service ticket을 수락하고 NFS 요청 처리
NAS/storage	Samba/winbind/idmap	Kerberos 이름을 AD RFC2307 UID/GID로 해석하여 파일 권한 판정

3. 사용자 keytab 발급 흐름

keytab은 컨테이너 안에서 만들지 않는다. AD DC에서 export한 뒤 target 계산 호스트의 root-only 경로에 설치한다.

```
sequenceDiagram
    participant CLI as 관리 서버 uidctl
    participant AD as Samba AD DC
    participant Host as target 계산 호스트
    participant NAS as NAS/storage
    participant Docker as Docker container

    CLI->>AD: 사용자·그룹 생성/조회
    CLI->>AD: uidNumber, gidNumber, group membership 설정
    CLI->>AD: domain exportkeytab --principal=user@REALM
    AD-->>CLI: 임시 keytab
    CLI->>Host: /etc/decs-krb/keytabs/user.keytab<br/>root:root 0400 설치
    CLI->>NAS: home 생성 및 numeric UID:GID 준비
    CLI->>Host: refresh env/service/timer 설치·활성화
    Host->>AD: kinit -k -t user.keytab
    AD-->>Host: 사용자 TGT가 든 ccache
    Host->>Host: /run/user/uid/krb5cc로 원자 교체
    CLI->>Host: 실제 NFS 쓰기 검증
    CLI->>Docker: ccache dir와 krb5.conf만 bind mount
```

실제 생성 구현은 다음 순서로 동작한다.

1. DB/기존 AD/storage 값을 사용해 UID/GID를 선택한다.
2. AD user/group과 RFC2307 `uidNumber`, `gidNumber` 를 보장한다.
3. AD DC에서 사용자 keytab을 임시 파일로 export하고 `0400` 으로 만든다.
4. target이 다른 host이면 Ansible `fetch` 와 `copy` 로 root-only keytab을 옮긴다.
5. NAS/storage home을 동일 UID/GID로 준비한다.
6. target host에 ccache directory, refresh helper, service/timer를 만든다.
7. host에서 사용자 ccache를 발급하고 NFS owner와 실제 write/delete를 검증한다.
8. 검증 후에만 컨테이너와 DB record를 확정한다.

관련 코드는 [AD identity](#)와 [keytab 생성](#), [container 생성 transaction](#), [credential 경로 모델](#)에서 확인한다.

4. 실제 NFS 인증 흐름

keytab 발급과 매 파일 접근의 인증은 서로 다른 흐름이다. 평상시 파일 I/O에서 컨테이너가 keytab을 직접 읽는 일은 없다.

```
sequenceDiagram
    autonumber
    participant P as container 사용자 process
    participant K as host kernel NFS client
    participant G as host rpc.gssd
    participant KDC as AD KDC
    participant S as NAS svcgssd/NFS
    participant ID as AD RFC2307 / winbind

    P->>K: open/read/write (호출 UID 포함)
    K->>G: RPCSEC_GSS credential upcall
    G->>G: /run/user/uid/krb5cc에서 사용자 TGT 확인
    G->>KDC: nfs/nas.farm.decs.internal service ticket 요청
    KDC-->>G: NFS service ticket
    G-->>K: GSS security context
    K->>S: NFSv4 RPC + Kerberos credential
    S->>S: service keytab으로 ticket 복호화
    S->>ID: principal을 RFC2307 UID/GID로 해석
    ID-->>S: numeric UID/GID와 group
    S-->>K: 파일 권한 판정 결과
    K-->>P: I/O 결과
```

인증 실패 위치에 따라 증상이 달라진다.

실패 위치	대표 증상
사용자 keytab/ccache	<code>kinit</code> 또는 <code>klist</code> 실패, 사용자 접근만 <code>Permission denied</code>
DNS/FQDN/SPN	<code>kvno nfs/<fqdn></code> 실패, IP source mount에서 principal 불일치
host machine keytab/ <code>rpc.gssd</code>	부팅 시 mount 실패, readiness의 keytab/kinit/rpc-gssd stage 실패
NAS service keytab/KVNO	여러 client가 동시에 GSS 실패, <code>kvno -k</code> 복호화 실패
RFC2307/idmap	인증은 되지만 owner가 <code>nobody</code> 이거나 UID/GID가 달라 쓰기 거부
NFS transport/kernel	<code>not responding</code> , D-state, mount probe timeout

5. Keytab과 ccache를 분리한 이유

구분	keytab	ccache
의미	principal의 장기 비밀키	이미 발급된 TGT/service ticket 묶음
새 ticket 발급	가능	제한된 renew 기간 안에서만 가능

구분	keytab	ccache
유출 영향	rotation할 때까지 반복 악용 가능	ticket lifetime/renew lifetime에 제한
저장 위치	host root-only	/run/user/<uid>/krb5cc, 사용자 0600
container 전달	금지	bind mount하여 전달

컨테이너 삭제만으로 유출된 keytab을 무효화할 수 없기 때문에 image, Docker volume, Kubernetes Secret에 사용자 keytab을 그대로 넣지 않는다. 호스트 refresh service는 임시 ccache에서 `klist` 검증을 끝낸 뒤 소유권 `uid:gid`, mode `0600`을 적용하고 최종 경로로 원자 교체한다.

현재 코드가 구현하는 대상은 Docker container다. Kubernetes Pod도 같은 원칙을 적용해야 하지만, Pod가 다른 node로 이동할 수 있으므로 단순 hostPath와 특정 node의 systemd timer에 의존해서는 안 된다. Pod 지원을 추가할 때는 node 배치, credential 발급 controller/CSI, rotation과 Pod 종료 시 정리를 별도 설계해야 한다.

6. Ticket 갱신 설계

`decs-krb-refresh@<username>.timer`는 기본적으로 boot 2분 뒤 시작하고 사용자별 설정 주기(현재 일반적으로 1시간)마다 oneshot service를 실행한다.

유효 ccache 있음

- └─ renew deadline이 margin보다 멀 -> 임시 복사본에서 `kinit -R`
- └─ deadline 임박/renew 실패 -> root-only keytab으로 새 ticket 발급

발급 결과 -> `klist` 검증 -> `uid:gid 0600` -> 최종 ccache로 atomic move

기본 재발급 여유는 `DECS_KRB_REISSUE_BEFORE_SECONDS=86400` (24시간)이다. 기존 ticket의 PAC/group 정보는 renew 시 유지될 수 있으므로 AD group 변경 뒤에는 keytab을 이용한 fresh login을 한 번 강제해야 한다.

7. NFS service identity와 KVNO

FARM NFS SPN은 Synology domain member의 machine account `NAS$`가 아니라 전용 AD service account `svc-nfs-farm`이 소유한다.

```
nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
nfs/NAS@FARM.DECS.INTERNAL
```

과거에는 `NAS$` machine password가 주기적으로 변경될 때 KVNO도 바뀌어 NAS의 정적 NFS keytab과 어긋날 수 있었다. 이를 service identity와 domain membership의 lifecycle 결합 문제로 보고 NFS SPN을 전용 계정으로 분리했다.

현재 `svc-nfs-farm`에는 자동 password expiration/rotation timer가 없다. 따라서 “NAS KVNO가 지금도 주기적으로 바뀐다”가 아니라 과거 자동 변경 문제를 전용 계정으로 제거했고, 현재 KVNO는 관리자가 명시적으로 rotation할 때만 바뀐다가 정확한 운영 모델이다. 별도 5분 checker는 변경을 만들지 않고 AD KVNO와 두 NAS keytab의 일치만 확인한다.

NAS의 `svcgssd`는 FARM과 AILAB principal을 한 keytab에서 받아들이므로 `-p`로 FARM principal 하나만 강제하지 않는다. keytab repair 시에도 AILAB acceptor를 보존한다.

8. FQDN mount와 service principal

Kerberos의 NFS service identity는 host-based principal이다. FARM mount source는 다음처럼 principal의 hostname과 같아야 한다.

```
mount source: nas.farm.decs.internal:/volume1/share
service SPN:  nfs/nas.farm.decs.internal@FARM.DECES.INTERNAL
transport:    addr=100.100.100.120
```

`100.100.100.120:/volume1/share`를 source로 쓰면 client가 IP 기반 service identity를 찾으려 하거나 기대한 FQDN SPN과 연결하지 못할 수 있다. 반면 source를 FQDN으로 유지한 상태에서 runtime option에 `addr=100.100.100.120`이 보이는 것은 정상이다. 관리 SSH 주소 `192.168.2.30`은 NFS transport로 사용하지 않는다.

9. 보안 flavor와 권한 모델

- `sec=krb5`: 사용자/서비스 인증. RPC payload 무결성·암호화 wrapping 없음.
- `sec=krb5i`: 인증 + RPC payload 무결성.
- `sec=krb5p`: 인증 + 무결성 + privacy encryption.

현재 내부 FARM/LAB 기본은 `sec=krb5`다. packet capture에 payload가 포함될 수 있으므로 NFS forensics 산출물은 root-only로 보관한다.

Kerberos 인증 뒤 최종 파일 권한은 numeric UID/GID와 mode/ACL로 결정된다. 컨테이너 root가 host credential 경계를 우회하지 못하도록 Kerberos mode에서는 `DECS_USER_SUDO_MODE=restricted`를 함께 사용한다. 다만 restricted sudo는 완전한 sandbox가 아니므로 강한 격리가 필요하면 sudo와 `SYS_ADMIN`, host bind mount 범위도 별도로 줄여야 한다.

10. 설정과 구현 위치

관심사	기준 코드/문서
FARM endpoint, SPN, mount option, NAS keytab	FARM runbook
LAB topology와 rollout gate	LAB runbook

관심사	기준 코드/문서
사용자 AD/keytab/ccache 명령 생성	commands.py
keytab·ccache·home 경로	paths.py
create-container transaction과 Docker bind	create_container.py
container credential 대기와 restricted sudo	entrypoint.sh
host machine keytab/readiness/fstab	kerberos_nfs_client_recovery.yml
NFS GSS readiness와 recovery	cluster-monitor-exporter scripts
NAS service keytab drift checker	check-nfs-keytab.sh

Kerberos/NFS 운영

이 문서는 현재 구현된 Docker/FARM·LAB 흐름의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.
5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run` 으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab` 을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber` 가 선택한 DB UID/GID와 일치한다.
- target host keytab이 `root:root 0400` 이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 해당 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf` 만 전달된다.

구현은 `create_container.py`와 `Kerberos command builder`에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \
  /etc/decs-krb/keytabs/<username>.keytab \
  /etc/decs-krb/refresh.d/<username>.env

sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400
refresh env: root:root 600
ccache dir:  <uid>:<gid> 700
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager

sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \
  klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하는 것이 현재 구현이다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |
  grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)='

docker exec --user <username> <container> \
  sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4
defaults,vers=4.0,rsize=1048576,wsize=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,
_netdev,exec,nouser 0 0
```

핵심은 source가 FQDN이라는 점이다.

```
올바름: nas.farm.decs.internal:/volume1/share
잘못됨: 100.100.100.120:/volume1/share
잘못됨: 192.168.2.30:/volume1/share
정상: mount option 또는 findmnt 결과의 addr=100.100.100.120
```

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal과 이름이 맞지 않는다. `192.168.2.30` 은 NAS 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
findmnt -T /home/tako8/share -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n '/\/home\/tako8\/share/,+4p'
getent hosts nas.farm.decs.internal
```

desired fstab 변경은 [server-state playbook](#)이 소유한다. 이미 mount된 legacy superblock은 유지보수 창에 clean unmount/remount한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다.

```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')
sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "${short}@FARM.DECS.INTERNAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd`를 `-n`으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n`은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. NAS service account와 KVNO 운영

현재 정책

- NFS SPN 소유자: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```

cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'

```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재

- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 FARM config에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair` 는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd` 를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

8. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery

무엇을 확인하는가	대표 상태/지표	코드
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

9. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME`, ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

10. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt`, `klist`, checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

11. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Monitoring

원본: md/system/monitoring/index.md, md/system/monitoring/design.md, md/system/monitoring/operations.md

역할: FARM/LAB 서버, NFS, GPU와 사용자 container의 실행 상태를 반복 관측하고, Prometheus·Grafana·Alertmanager를 통해 시각화와 경보를 제공한다.

이 문서는 monitoring 문서의 시작점이다. **설계**는 수집 데이터가 어떤 component를 거쳐 metric과 alert가 되는지 설명하고, **운영**은 배포·점검·장애 대응 명령과 코드 위치를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위, 핵심 component와 운영 원칙
설계	exporter, Prometheus, Grafana, Alertmanager, GSS health와 forensics의 데이터 흐름
운영	배포, endpoint 점검, metric/alert 코드 링크, 장애별 진단 순서

한눈에 보는 구조



핵심 component

component	endpoint/산출물	역할
gpu-user-exporter	:30072/metrics, :30072/-/healthy	GPU process를 DB의 실제 사용자/container에 귀속
cluster-monitor-exporter	:30074/metrics, :30074/healthz	mount, GSS, GPU, Docker, container, 연결성, D-state 수집
public health listener	서버별 N89/healthz	외부에서 NAT와 host 도달 가능성 확인
NFS GSS worker	/run · /var/lib state	readiness, canary와 guarded missing-mount recovery
NFS forensics	status JSON, local incident snapshot	제한된 packet/kernel trace를 보존
keytab health check	profile status JSON	AD KVNO, storage keytab과 GSS service drift를 읽기 전용 확인
Prometheus/Alertmanager	cluster별 Kubernetes release	scrape, rule 평가, alert routing
Grafana	FARM NodePort 30080	FARM/LAB datasource와 dashboard를 한 UI에서 제공

운영 경계

- 지속 관측은 **monitoring**, 부팅 orchestration은 **remote-operations**, host desired state는 **server-state**가 소유한다.
- cluster-monitor-exporter**가 직접 mount하지 않는다. 별도 systemd recovery worker가 fstab, DNS/RPC, Kerberos readiness, canary와 D-state gate를 통과한 **missing mount**만 복구한다.
- keytab checker는 AD password, NAS keytab, service와 mount를 변경하지 않는다.
- forensics는 증거를 수집할 뿐 mount/restart/reboot를 수행하지 않는다.
- 자동 복구는 stopped container 시작, SSH 시작, 안전성이 확인된 NVML symlink repair처럼 영향 범위가 제한된 작업에만 사용한다.
- FARM과 LAB Prometheus는 장애와 저장소를 분리하고, Grafana dashboard만 공통 UI에서 datasource UID와 cluster label로 구분한다.

주요 코드

- cluster-monitor-exporter
- gpu-user-exporter
- Prometheus/Alertmanager 설정
- Grafana dashboards
- monitoring Ansible

- [NFS forensics](#)
- [Kerberos/NFS keytab checker](#)

Kerberos credential과 FQDN mount 원칙은 [Kerberos/NFS 설계](#), KVNO·timer·mount 장애 절차는 [Kerberos/NFS 운영](#)을 함께 참고한다.

Monitoring 설계

이 문서는 host의 원시 상태가 metric, alert와 dashboard가 되기까지의 흐름과 자동 복구의 안전 경계를 설명한다.

1. 설계 목표

- FARM/LAB host, GPU, container와 storage 상태를 동일한 metric 체계로 관측한다.
- HTTP process가 살아 있는지와 실제 collection이 갱신되는지를 구분한다.
- NFS **hard** mount와 D-state 상황에서도 monitoring loop 전체가 멈추지 않게 한다.
- alert에는 원인 진단에 필요한 stage와 label을 포함하되 cardinality를 제한한다.
- 관측, 증거 수집과 상태 변경을 분리하여 monitoring이 장애 반경을 넓히지 않는다.
- FARM/LAB Prometheus failure domain은 분리하고 공통 Grafana에서 함께 조회한다.

2. 전체 데이터와 알람 흐름

flowchart TB

subgraph H["FARM/LAB host"]

SRC["/proc · mountinfo · systemd
nvidia-smi · Docker · journal"]

GPU["gpu-user-exporter
:30072"]

CM["cluster-monitor-exporter
:30074 / public :N89"]

GSS["GSS readiness · canary
mount recovery worker"]

FOR["NFS trace ring · watcher
snapshot worker"]

KEY["keytab profile checker
user systemd timer"]

SRC --> GPU

SRC --> CM

GSS -->|"state file"| CM

FOR -->|"status JSON"| CM

end

DB["UID DB"] -->|"container owner cache"| GPU

AD["AD / storage"] -. ->|"read-only KVNO/keytab check"| KEY

GPU -->|"Prometheus metrics"| PF["FARM Prometheus"]

CM -->|"Prometheus metrics"| PF

GPU -->|"Prometheus metrics"| PL["LAB Prometheus"]

CM -->|"Prometheus metrics"| PL

PF --> GRAF["FARM Grafana"]

PL -->|"prometheus-lab datasource"| GRAF

PF --> AM["Alertmanager"]

PL --> AM

AM --> RELAY["localhost notify relay"]

RELAY --> API["internal Slack notify API"]

EXT["Google Apps Script"] <-->|"public health / heartbeat"| CM

데이터 plane과 control plane

종류	예	설계 원칙
관측 data plane	/proc, nvidia-smi, Docker inspect, mountinfo, state JSON	bounded timeout, read-only 우선
metric plane	exporter /metrics → Prometheus	scrape 실패와 collection stale을 별도 표시
alert plane	Prometheus rule → Alertmanager → relay	secret은 Kubernetes Secret, alert label로 credential 전달 금지
visualization plane	FARM/LAB datasource → Grafana	datasource UID와 cluster label로 환경 구분

종류	예	설계 원칙
제한된 control plane	container start, SSH start, NVML repair, missing mount worker	명시적 enable, 사전조건, retry/inhibit 필요

3. cluster-monitor-exporter

Background collection과 freshness

exporter는 HTTP 요청 때마다 무거운 host 명령을 실행하지 않고 background loop의 마지막 결과를 text format으로 제공한다. `/healthz` 는 process 응답뿐 아니라 마지막 성공 collection이 `metrics_stale_after` 이내인지 확인한다.

이 구분이 필요한 이유는 HTTP server는 살아 있지만 child command나 collection goroutine이 멈춘 상태를 `up=1` 만으로는 찾을 수 없기 때문이다. 대표 지표는 `cluster_monitor_exporter_last_collection_timestamp_seconds` 와 `cluster_monitor_exporter_metrics_stale_after_seconds` 다.

수집/렌더링 구현은 `main.go`에서 확인한다.

관측 범위

- required mount의 source/target/filesystem/security option과 응답 시간
- storage host 및 peer 연결성, mount failure diagnosis
- 모든 thread를 포함한 host D-state process와 command별 count
- kernel hung-task 증가, NFS lease와 forensic 상태
- host `nvidia-smi` , Docker daemon
- 대상 image container의 running/SSH/GPU readiness
- NVML driver/library mismatch와 제한된 repair 결과
- 외부 connectivity heartbeat와 public inbound health
- NFS GSS readiness/canary/recovery state와 `rpc-gssd` journal

D-state와 명령 timeout

NFS `hard` mount에서 child process가 D-state에 들어가면 `SIGKILL` 에도 즉시 끝나지 않고 `cmd.Wait()` 가 무기한 block될 수 있다. exporter는 외부 명령마다 timeout을 두고 collection loop가 한 NFS probe 때문에 완전히 멈추지 않게 한다.

D-state는 발생 즉시 total과 command별 metric으로 기록하되 alert는 30분 연속 지속을 기다린다. `sshd` , `find` , `du` , `chown` , `chmod` , `codex` , `claude` , `python` , `python3` 은 0인 시계열도 유지하고, 다른 command는 실제 관측될 때만 동적으로 노출한다. hung-task alert는 과거 누적 line이 아니라 최근 5분 증가분을 사용한다.

제한된 container 복구

설정으로 허용된 경우에만 다음을 수행한다.

- stopped target container 시작

- container 내부 SSH service 시작
- host driver version 검증 뒤 `libnvidia-ml.so.1` symlink repair

임의 image/container, user data, host mount를 변경하지 않는다. 복구 시도와 결과는 별도 metric으로 남겨 “정상”과 “복구되어 정상”을 구분한다.

4. gpu-user-exporter

`nvidia-smi` 는 PID와 GPU 사용량만 제공하므로 실제 사용자와 container를 알기 위해 여러 정보를 join한다.

flowchart LR

```
SMI["nvidia-smi<br/>GPU/PID/memory/util"] --> JOIN["gpu-user-exporter"]
PROC["/proc/PID/cgroup<br/>container ID"] --> JOIN
DOCKER["Docker inspect<br/>state/device request"] --> JOIN
DB["UID DB cache<br/>owner/real name"] --> JOIN
JOIN --> METRIC["사용자-container-GPU별<br/>memory/util/process metric"]
```

주요 지표:

- `docker_gpu_user_memory_used_bytes`
- `docker_gpu_user_sm_utilization_percent`
- `docker_gpu_user_process_count`
- device 전체 utilization/memory
- ignored process count
- DB cache entry/refresh success
- exporter scrape success/duration

DB에 없는 process를 임의 사용자에게 귀속하지 않고 ignored count로 분리한다. 매 scrape마다 DB를 조회하지 않도록 활성 container owner를 cache하고 refresh 성공과 cache 크기를 metric으로 노출한다. 구현은 [gpu-user-exporter main.go](#)에 있다.

5. NFS GSS readiness, canary와 recovery

Readiness

부팅 시 NFS mount가 Kerberos host identity보다 먼저 실행되는 race를 막기 위해 다음 stage를 순서대로 확인한다.

```
host keytab -> machine kinit -> NFS service kvno
-> rpc_pipefs -> rpc-gssd -> (환경별 canary)
```

각 stage와 diagnosis는 state file에 기록되고 exporter가 metric으로 변환한다. 구현은 [readiness script](#)와 [NFS GSS collector](#)에 있다.

Canary

canary는 실제 user share가 아니라 전용 read-only export에서 service path를 확인한다. LAB은 canary를 사용하며 FARM은 전용 export가 준비되기 전까지 canary gate만 비활성화하고 나머지 readiness를 유지한다.

Guarded missing-mount recovery

exporter는 recovery state를 관측하고 loopback API로 worker를 queue할 수 있지만 실제 `mount` 는 별도 systemd service가 수행한다. worker gate는 다음과 같다.

1. 기대 source/filesystem/security로 이미 mount되어 있으면 즉시 healthy로 종료한다.
2. mount가 없을 때만 fstab 기대 entry를 확인한다.
3. NFS/RPC D-state가 있으면 새 mount 시도를 보류한다.
4. DNS와 storage RPC를 확인한다.
5. Kerberos readiness를 확인한다.
6. 환경에서 요구하는 canary를 확인한다.
7. retry/backoff/inhibit가 허용할 때만 `mount <target>` 을 실행한다.

healthy mount를 unmount/remount하지 않으며, mount timeout이나 관련 D-state가 생기면 현재 boot에서 반복 시도를 inhibit할 수 있다. 구현은 [mount recovery script](#)에 있다.

6. NFS forensics

NFS 장애는 재시작 후 증거가 사라질 수 있으므로 recovery와 독립된 passive forensics를 둔다.

- TCP/2049 packet의 크기 제한 ring
- 저용량 ftrace instance
- 5초 간격 kernel/NFS/D-state 상태 관측
- incident threshold에서 pre-incident ring을 local snapshot으로 보존
- PID와 process start time을 함께 사용해 PID 재사용으로 duration이 이어지지 않게 함

forensics는 mount/unmount, user share 읽기, NFS/GSS restart, recovery, reboot를 하지 않는다. exporter는 `/run/decs-nfs-forensics/status.json` 만 읽는다. `sec=krb5` 는 payload privacy를 제공하지 않으므로 capture와 incident directory는 root-only다.

구현은 [trace buffer](#), [watcher](#), [snapshot](#), [metric adapter](#)에서 확인한다.

7. Keytab health check

AD/NAS service keytab은 exporter의 자동 복구 대상이 아니다. 별도 user systemd timer가 profile별 checker를 실행한다.

```
AD KVNO + required SPN
vs storage keytab KVNO/principal
+ 실제 service ticket 복호화
+ GSS service process/forbidden option
-> profile status JSON
```

exit code는 정상 0, 점검 오류 1, drift 2 다. keytab이나 credential 원문은 status에 기록하지 않는다. 코드는 [check-nfs-keytab.sh](#)에 있다.

8. Prometheus, Alertmanager와 Grafana

Cluster 분리

- FARM: `kube-prometheus-stack`, Prometheus/Grafana local PV, 공통 Grafana 제공
- LAB: LAB8 control plane의 독립 Prometheus, NodePort 30073
- Grafana: LAB datasource UID `prometheus-lab` 으로 LAB dashboard 조회

retention, storage, target, Alertmanager routing과 control-plane failure domain이 다르므로 Prometheus release와 values를 분리한다. dashboard는 운영자가 한 UI에서 비교하므로 공통 디렉터리에 둔다.

Alert delivery

Prometheus rule은 Alertmanager로 전달되고, Alertmanager는 localhost relay sidecar에 native payload를 보낸다. relay가 내부 notify API contract로 변환하여 `/api/internal/slack/notify` 로 전달한다. Grafana password와 Slack webhook은 values가 아니라 Kubernetes Secret으로 관리한다.

canonical FARM rule과 routing은 `prometheus-farm-values.yaml`, relay 구현은 Alertmanager relay ConfigMap에서 확인한다.

9. 설정과 코드 위치

관심사	기준 파일
exporter 공통 기본값, FQDN mount/SPN, GSS/forensics option	<code>group_vars/exporters.yml</code>
exporter build/install/systemd config	<code>deploy_exporters.yml</code>
GSS readiness/canary/recovery 설치	<code>deploy_nfs_gss_health.yml</code>
forensics 설치	<code>deploy_nfs_forensics.yml</code>
FARM Prometheus/Alertmanager rules	<code>prometheus-farm-values.yaml</code>
LAB Prometheus values	<code>prometheus-lab-values.yaml</code>

관심사	기준 파일
GPU/NIC/storage dashboards	grafana/dashboards

10. 설계 변경 시 확인할 항목

- 새 metric의 이름, type, help, label cardinality와 실패 시 값
- command timeout과 NFS D-state에서 collection loop가 유지되는지
- alert **for** 시간이 일시적 부팅/배포 지연보다 충분히 긴지
- recovery의 enable flag, precondition, retry/backoff, inhibit와 audit metric
- user data를 읽지 않는 canary/probe 경로
- FARM/LAB datasource, job, label과 dashboard query의 일치
- credential, webhook, DB password가 Git/metric/label/log에 노출되지 않는지

Monitoring 운영

이 문서는 monitoring component를 배포하고 endpoint, metric, alert와 incident를 점검하는 방법을 설명한다.

1. 배포 순서

새 host 또는 NFS GSS monitoring을 처음 적용할 때는 다음 순서를 권장한다.

1. inventory와 host FQDN/SPN/mount source 확인
2. NFS GSS readiness/canary/recovery worker 설치
3. NFS forensics 설치
4. exporter 배포
5. Prometheus target/rule 반영
6. Grafana datasource/dashboard 확인
7. alert delivery test

운영 배포 entry point는 exporter 내부 개별 shell script가 아니라 [monitoring/ansible_playbook/](#) 이다.

2. Exporter 배포

```
cd /home/jy/server_manage/monitoring
```

두 exporter를 한 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=farm8
```

FARM 전체 또는 FARM/LAB 전체:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=FARM
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml
```

한 exporter만 전체 재배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_gpu_user_exporter_all.yml
```

target에는 다음 systemd service와 endpoint가 생긴다.

```
cluster-monitor-exporter.service -> :30074/metrics, :30074/healthz, :N89/healthz  
gpu-user-exporter.service        -> :30072/metrics, :30072/-/healthy
```

배포 기준과 요구 조건은 [Ansible README](#), 실제 task는 [deploy_exporters.yml](#)에서 확인한다.

3. NFS GSS와 forensics 배포

NFS GSS readiness/canary/recovery worker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \  
-e nfs_gss_health_hosts=lab8
```

NFS forensics:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

중요한 systemd unit과 state:

```
decs-kerberos-nfs-ready.service  
decs-nfs-gss-canary-probe.service/.timer  
decs-nfs-gss-mount-recovery.service/.timer  
/run/decs-nfs-gss/ready.state  
/var/lib/decs-nfs-gss/canary.state  
/var/lib/decs-nfs-gss/recovery.state  
  
decs-nfs-forensics-buffer.service  
decs-nfs-forensics-watch.service/.timer  
decs-nfs-forensics-snapshot.service  
/run/decs-nfs-forensics/status.json  
/var/lib/decs-nfs-forensics/
```

FARM은 전용 read-only canary export가 준비될 때까지 canary만 비활성화하며 keytab/kinit/kvno/rpc-gssd readiness와 guarded recovery는 유지한다. LAB canary는 `sec=krb5` 전용 read-only export를 사용한다.

4. Keytab checker 배포

FARM 읽기 전용 checker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml
```

LAB을 상시 점검 대상으로 전환한 뒤에만 두 profile을 활성화한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml \  
-e '{"keytab_check_profiles":["farm","lab"]}'
```

```
systemctl --user list-timers 'check-nfs-keytab@*.timer'  
systemctl --user status check-nfs-keytab@farm.service --no-pager
```

checker는 drift를 고치지 않는다. KVNO history, repair와 rotation은 [Kerberos/NFS 운영](#)을 따른다.

5. Prometheus/Alertmanager 배포

먼저 server-side render와 cluster precondition만 검증한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml
```

변경 적용:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml \  
-e prometheus_dry_run=false
```

한 환경만 배포하려면 `prometheus_farm_enabled=false` 또는 `prometheus_lab_enabled=false` 를 사용한다. playbook은 kubeconfig 대상 cluster, node Ready/DiskPressure, 필수 Secret 이름을 확인한 후 Helm을 실행한다.

FARM에서 필요한 Secret:

- `monitoring-grafana-admin`: `admin-user`, `admin-password`
- `cluster-monitor-slack-webhook-farm`: `url`

값은 Git values에 넣지 않는다. Alertmanager는 Slack webhook으로 직접 보내지 않고 localhost relay를 거쳐 internal notify API로 보낸다.

6. 배포 후 endpoint 확인

target host에서:

```
systemctl status cluster-monitor-exporter gpu-user-exporter --no-pager  
journalctl -u cluster-monitor-exporter -n 100 --no-pager  
journalctl -u gpu-user-exporter -n 100 --no-pager  
  
curl -fsS http://127.0.0.1:30074/healthz  
curl -fsS http://127.0.0.1:30074/metrics | head  
curl -fsS http://127.0.0.1:30072/-/healthy  
curl -fsS http://127.0.0.1:30072/metrics | head
```

public health port는 서버 번호 `N` 에 대해 `9000 + N*100 - 11` 이다. 예를 들어 FARM8은 `9789/healthz` 다. 이 endpoint는 exporter process/collection freshness와 외부 NAT 도달 경로를 확인하는 용도이지, 모든 dependency의 readiness를 대신하지 않는다.

NFS GSS host-only API:

```
curl -fsS http://127.0.0.1:30074/nfs-gss/ready
curl -fsS http://127.0.0.1:30074/nfs-gss/health
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/probe
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/recover
```

probe 와 **recover** 는 loopback 요청만 허용하며 worker를 queue한 뒤 즉시 응답한다. **recover** 를 반복 호출하기 전에 recovery state와 inhibit reason을 확인한다.

7. 무엇을 어디에서 보는가

관측 대상	대표 metric/alert	구현/설정 링크
exporter process와 freshness	<code>up</code> , <code>cluster_monitor_exporter_last_collection_timestamp_seconds</code> , <code>ClusterMonitorExporter*</code>	collector, rules
required mount와 latency	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , <code>..._probe_seconds</code>	mount collector, storage dashboards
Kerberos NFS readiness	<code>cluster_monitor_nfs_gss_ready</code> , <code>ClusterMonitorNFSGSSReadinessFailed</code>	readiness, GSS collector
canary/recovery	<code>cluster_monitor_nfs_gss_canary_*</code> , <code>..._recovery_*</code>	canary, recovery
D-state, lease, hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code> , <code>ClusterMonitorNFS*</code>	forensics adapter, forensics scripts
host GPU/Docker/container	<code>cluster_monitor_host_gpu_up</code> , <code>...docker_daemon_up</code> , <code>...container_*</code>	cluster collector
사용자별 GPU	<code>docker_gpu_user_memory_used_bytes</code> , <code>...sm_utilization_percent</code> , <code>...process_count</code>	gpu-user-exporter, GPU dashboard
외부 연결	<code>cluster_monitor_external_connectivity_*</code> , public health	exporter, Apps Script
AD/NAS keytab	checker JSON/exit code	keytab checker

canonical alert rule은 [prometheus-farm-values.yaml](#)이다. exporter 내부 `prometheus/cluster_monitor_alerts.yml` 은 metric 이름을 보여 주는 reference이며 실제 FARM release와 값이 다를 수 있다.

8. Alert별 진단 순서

Exporter down 또는 stale

1. Prometheus target에서 connection error와 scrape timestamp를 구분한다.

2. target systemd status/journal을 확인한다.
3. `:30074/healthz` 와 `/metrics` 를 각각 확인한다.
4. process는 살아 있지만 stale이면 마지막 실행 command와 D-state를 확인한다.
5. 변경된 env/config와 binary 배포 시간을 확인한 뒤 exporter만 재배포한다.

Required mount down/slow/unresponsive

1. `findmnt` 로 mount 존재, FQDN source, filesystem과 `sec` 를 확인한다.
2. mount probe와 storage ping/peer diagnosis label을 확인한다.
3. GSS readiness에서 처음 실패한 stage를 확인한다.
4. D-state, lease expiry, hung-task와 local forensic snapshot을 확인한다.
5. mount가 없을 때만 recovery state를 확인한다.
6. mount가 이미 있거나 D-state caller가 있으면 강제 remount를 반복하지 않는다.

Kerberos source는 IP가 아니라 FQDN이어야 한다. 자세한 기준은 [Kerberos/NFS 운영의 mount 절](#)을 따른다.

GSS readiness/canary/recovery

```
systemctl status decs-kerberos-nfs-ready.service --no-pager
systemctl status decs-nfs-gss-canary-probe.timer --no-pager
systemctl status decs-nfs-gss-mount-recovery.timer --no-pager
journalctl -u decs-kerberos-nfs-ready.service -n 100 --no-pager
cat /run/decs-nfs-gss/ready.state
cat /var/lib/decs-nfs-gss/canary.state
cat /var/lib/decs-nfs-gss/recovery.state
```

- keytab/kinit stage: host machine credential 확인
- kvno stage: DNS/FQDN/SPN/KDC 확인
- rpc-gssd stage: service와 잘못된 `-n` override 확인
- canary stage: 전용 export와 user share를 혼동하지 않았는지 확인
- inhibited recovery: D-state 또는 mount timeout 증거를 보존하고 원인을 처리

GPU 사용자 mapping 이상

1. `nvidia-smi` 와 exporter scrape success를 확인한다.
2. ignored process count가 증가했는지 확인한다.
3. PID의 cgroup container ID와 Docker inspect를 비교한다.
4. DB cache refresh success와 cache age/entry를 확인한다.
5. DB record와 실제 running container가 어긋났다면 user-lifecycle 소유 절차로 고친다.

Container SSH/GPU alert

1. container가 running인지 먼저 확인한다.
2. exporter가 start/SSH/NVML recovery를 시도했는지 metric과 journal을 확인한다.
3. container image가 configured regex 대상인지 확인한다.
4. NVML mismatch가 아니라 application/GPU allocation 문제면 자동 symlink repair를 반복하지 않는다.

9. Grafana 점검

- LAB dashboard datasource UID가 `prometheus-lab` 인지 확인한다.
- dashboard query의 `cluster`, `server_id`, `job` label이 scrape config와 일치하는지 확인한다.
- storage dashboard에서 mount probe, responsive, D-state, blocked process, hung-task를 같은 시간축으로 비교한다.
- GPU dashboard에서 DB-known 사용자 metric과 device 전체 metric을 함께 본다.
- 새 server 추가 시 dashboard에 hard-coded server/GPU panel 누락이 없는지 확인한다.

dashboard 원본:

- [FARM storage latency](#)
- [LAB storage latency](#)
- [GPU usage](#)
- [Network traffic](#)

10. 테스트

```
cd /home/jy/server_manage/monitoring

(cd prometheus/exporters/cluster-monitor-exporter && go test ./...)
(cd prometheus/exporters/gpu-user-exporter && go test ./...)
bash nfs-forensics/tests/test_watch.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
python3 prometheus/config/tests/test_slack_notify_relay.py
```

변경 위험에 맞게 최소한 다음을 함께 검증한다.

- metric 이름/type/help와 0/failure path
- recovery가 healthy mount를 건드리지 않는지
- D-state에서 mount attempt가 차단되지만 existing mount는 healthy로 처리되는지
- Alertmanager relay가 secret을 출력하지 않고 internal API payload로 변환하는지
- FARM/LAB dashboard datasource가 바뀌지 않았는지

11. 새 서버 추가 체크리스트

1. `ansible_playbook/inventory.ini` 에 `farmN/ LabN` 형식으로 추가
2. `group_vars/exporters.yml` 의 mount source, SPN과 환경 기본값 확인
3. host share target와 public `N89` port 계산 확인
4. GSS readiness/forensics/exporter 배포
5. Prometheus `:30070` , `:30072` , `:30074` scrape target 추가
6. 방화벽/NAT public health 확인
7. Grafana dashboard server/GPU panel 확인
8. alert rule label과 Alertmanager route 확인
9. endpoint와 실제 alert delivery 검증

12. 운영 문서에 계속 추가할 내용

설계와 운영을 분리한 뒤 다음 정보를 운영 문서에 누적하면 좋다.

- 서비스별 owner, endpoint, systemd/Kubernetes resource와 dependency
- metric별 정상 범위, alert `for` 와 runbook link
- 최근 배포 version/commit과 마지막 검증 시각
- FARM/LAB retention, PV 용량과 capacity threshold
- alert silence 승인/만료 기준과 false-positive 기록
- 자동 복구별 시도 횟수, inhibit 해제와 rollback 절차
- incident snapshot 보존 기간, 접근 권한과 개인정보 취급
- 새 server/metric/dashboard 추가 checklist

설계 문서에는 component 관계와 안전 경계를 유지하고, host별 현재 값과 일회성 incident 결과는 canonical config/runbook 또는 별도 evidence에 기록한다.

remote-operations 설계·운영 매뉴얼

원본: md/system/remote-operations.md

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

1. 책임 범위

remote-operations 는 management host의 remote-boot.service 가 실행하는 부팅 orchestration이다. Wake-on-LAN, 우선 서버 gate, 1회 host health check와 container post-check를 소유한다.

주기적인 mount/GPU/container 관측은 monitoring 에 있다. 이 경계를 나눈 이유는 부팅 시 복구와 상시 self-healing이 서로 다른 retry 정책과 장애 의미를 가지기 때문이다. 이 디렉터리에서 관리하는 systemd unit은 remote-boot.service 하나다.

2. 디렉터리 지도

경로	핵심 기능
script/run_remote_boot.sh	전체 부팅 흐름의 entry point
script/wake_targets.sh	target 선택과 WOL magic packet 전송
script/wait_for_priority_servers.sh	priority/remaining host health gate
script/check_server_boot_health.sh	mount, GPU, 임시 test container 점검
script/restart_all_remote_containers.sh	기존 container 시작과 SSH/GPU post-check
script/create_test_container.sh	health check용 임시 GPU container 생성
script/delete_test_container.sh	임시 container 정리
script/common.sh	config, target, Ansible, log, alert 공통 함수
script/install_remote_boot_service.sh	systemd unit 설치·활성화
script/dry_run_remote_boot.sh	WOL/health/container/full flow simulation
script/integration_smoke_test.sh	수동 Ansible/Docker/GPU 통합 확인
config/remote_boot.example.env	공개 가능한 설정 구조
config/remote_boot.local.env	실제 MAC·webhook 등을 담는 ignored 설정

3. 부팅 상태 머신

```
pre-delay
|
v
wake priority targets
|
v
priority health gate ---- failure -----> stop + alert
|
v
secondary delay
|
v
wake remaining targets
|
v
remaining health gate -- failure -----> record + alert
|
v
start stopped target containers
|
v
per-container SSH/GPU post-check
```

각 domain에서 서버 한 대를 먼저 깨우는 구체적인 이유는 **LAB1** 과 **FARM1** 에 사용자 관리 DB가 Docker container로 존재하기 때문이다. 이 DB는 사용자, UID/GID, 할당 port와 사용자 container record를 관리하는 기준 데이터이므로, 나머지 compute server보다 먼저 DB host를 기동해 관리 상태가 복구될 시간을 확보한다.

따라서 **LAB1** 과 **FARM1** 을 priority target으로 먼저 깨우고 health gate를 통과한 뒤 나머지 서버를 기동한다. 이렇게 하면 사용자 관리 DB가 준비되지 않은 상태에서 전체 서버와 사용자 container 운영이 동시에 시작되는 상황을 줄일 수 있다. priority gate를 끌 수는 있지만 기본값은 이 의존 순서를 보존한다.

4. 핵심 기능

4.1 target 정규화

설정은 FARM/LAB 전체 목록, 기본 대상과 priority 대상을 별도로 가진다. 명령행 target이 있으면 기본 대상을 덮어쓴다. **all**, domain group, 개별 **FARM1** / **LAB10** 을 동일한 parser로 해석하여 중복을 제거한다.

MAC address와 broadcast IP는 WOL에만 쓰고, 상태 확인과 원격 명령은 Ansible inventory를 사용한다. 네트워크 주소와 논리 server ID를 분리하면 SSH port나 주소가 바뀌어도 workflow 이름을 유지할 수 있다.

4.2 1회 host health check

`check_server_boot_health.sh` 는 target에 대해 다음을 확인한다.

- Ansible/SSH 도달성
- 필수 FARM/LAB share mount
- host `nvidia-smi`
- 임시 test container 생성
- container 내부 GPU와 필요한 home mount
- test 종료 후 container 삭제

실제 사용자 container 대신 고정된 health test identity와 image를 사용하는 이유는 특정 사용자 상태를 서버 health로 오판하지 않고, test artifact를 명확히 정리하기 위해서다.

4.3 기존 container 재시작과 post-check

선택된 서버의 stopped target container를 시작한 뒤 각 container의 SSH와 GPU를 제한 시간 동안 확인한다. post-check 실패는 container별로 기록하고 전체 stage 실패 정보에 포함한다.

container를 무조건 재생성하지 않는 이유는 사용자 홈과 DB record, 고정 포트, 실행 옵션의 권위가 `user-lifecycle`에 있기 때문이다. 부팅 모듈은 기존 container를 시작하고 준비 여부만 확인한다.

4.4 alert suppression과 로그

실패는 구성된 경우 내부 notify API를 통해 Slack으로 보내고, 그렇지 않으면 stub log에 남긴다. 동일 실패의 반복 알림을 줄이기 위한 alert state가 별도 디렉터리에 저장된다. `reset_remote_boot_alert_state.sh` 는 이 suppression 상태만 초기화한다.

service log, host health log와 alert state를 나눈 이유는 실행 이력과 알림 deduplication 상태를 독립적으로 보존하기 위해서다.

5. 설정 모델

설정은 `remote_boot.example.env` 를 복사하여 local 파일에서 관리한다.

```
cd /home/jy/server_manage/remote-operations
cp config/remote_boot.example.env config/remote_boot.local.env
```

주요 설정 그룹:

그룹	예
target	FARM/LAB 목록, 기본/priority target
순서/gate	pre-delay, gate timeout/poll, secondary delay

그룹	예
container	restart enable, timeout, post-check poll
network	Ansible inventory, broadcast IP, MAC address
health	필수 mount와 host share template
test container	image, UID/GID, mount, memory, runtime
logging/alert	log path, rotate count, notify API와 webhook

실제 MAC, password와 webhook은 `remote_boot.local.env`에만 둔다. example에는 변수 이름과 안전한 placeholder만 유지한다.

6. 사용법

target 확인과 수동 WOL:

```
cd /home/jy/server_manage/remote-operations
./script/wake_targets.sh --list-targets
./script/wake_targets.sh FARM1 LAB1
```

한 서버의 boot health:

```
./script/check_server_boot_health.sh --server-id FARM1
```

한 서버의 기존 container 시작/post-check:

```
./script/restart_all_remote_containers.sh FARM1
```

systemd 설치와 확인:

```
./script/install_remote_boot_service.sh
sudo systemctl start remote-boot.service
systemctl status remote-boot.service
journalctl -u remote-boot.service -b
```

7. dry-run과 검증

```
./script/dry_run_remote_boot.sh wake FARM1 LAB1
./script/dry_run_remote_boot.sh health FARM1
./script/dry_run_remote_boot.sh containers FARM1
./script/dry_run_remote_boot.sh full FARM1 LAB1
```

dry-run은 sleep, WOL, Ansible 변경과 Docker 작업 대신 계획을 기록한다. 설정 파싱, target 분할과 단계 순서를 production 영향 없이 검증하기 위한 공개 계약이다.

실제 연결을 확인할 때:

```
./script/integration_smoke_test.sh --scope priority
```

운영 전에는 최소한 다음을 확인한다.

- priority target이 실제 의존 순서와 맞는지
- MAC/broadcast IP와 Ansible host가 같은 물리 서버를 가리키는지
- health test image가 target GPU driver와 호환되는지
- 임시 container 이름이 실제 사용자 container와 충돌하지 않는지
- gate timeout이 정상 부팅 시간보다 충분한지
- 실패 알림에 비밀값이 포함되지 않는지

8. 실패 처리 원칙

- priority gate 실패 시 나머지 서버를 깨우지 않는 것이 기본이다.
- remaining server 일부 실패는 성공한 서버의 후속 작업과 실패 대상을 구분해 기록한다.
- test container는 성공/실패와 무관하게 정리를 시도한다.
- mount 또는 GPU 문제를 부팅 script에서 무제한 복구하지 않는다. 반복 상태는 **monitoring**으로 넘기고, Kerberos/NFS 고위험 복구는 해당 runbook을 따른다.
- 실제 사용자 container의 DB record나 Docker 옵션을 부팅 script에서 다시 만들지 않는다.

9. 새 서버 추가

1. `user-lifecycle/server_info/servers.jsonl` 과 Ansible inventory에 서버를 등록한다.
2. local env의 FARM/LAB target 목록과 MAC을 추가한다.
3. 필요한 경우 priority 여부와 필수 mount template를 정한다.
4. WOL dry-run과 개별 WOL을 확인한다.
5. 개별 boot health check를 실행한다.

6. container restart를 한 서버에 제한해 검증한다.
7. `monitoring`의 scrape target/exporter 배포를 별도로 완료한다.

10. 설계상 지켜야 할 경계

- systemd unit을 추가하기 전에 부팅 1회 책임인지 지속 monitor 책임인지 구분한다.
- 공통 shell helper를 수정하면 모든 stage의 dry-run과 alert redaction을 확인한다.
- 실제 secret을 example 또는 log에 쓰지 않는다.
- retry를 늘리는 것으로 storage/GSS 장애를 감추지 않는다.
- container 생성/삭제 비즈니스 규칙은 `user-lifecycle` CLI를 통해 수행한다.

server-state 매뉴얼

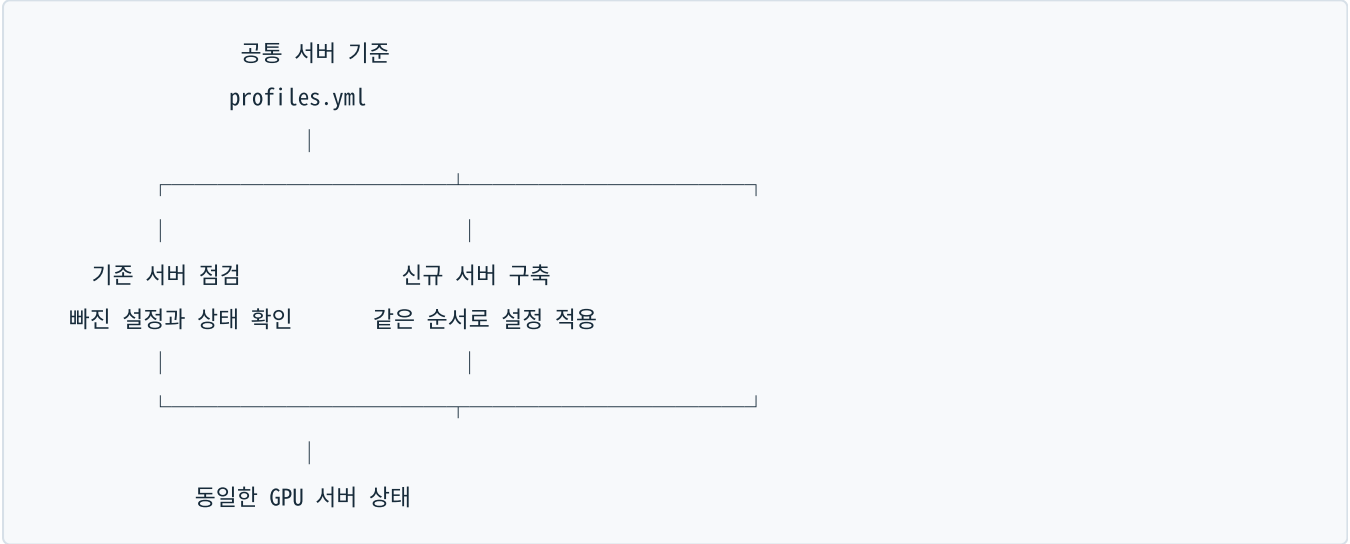
원본: md/system/server-state.md

1. 이 모듈이 하는 일

`server-state` 는 FARM/LAB의 GPU 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하는 모듈이다. 목적은 크게 두 가지다.

- 기존 서버 점검:** 각 서버에 Docker, NVIDIA driver/toolkit, Kubernetes, network tuning, Kerberos/NFS, monitoring 설정이 빠짐없이 적용되어 있는지 확인하고 차이가 있으면 복구할 방법을 제시한다.
- 신규 서버 구축:** 새 서버에 공통 설정을 같은 순서로 적용하여 기존 서버와 동일한 표준 상태로 만든다.

즉, 서버마다 설치 방법을 다시 기억해서 수동으로 설정하는 대신 하나의 공통 기준을 사용하려고 만든 코드다.



`server-state` 가 모든 기능을 직접 구현하는 것은 아니다. 공통 OS, Docker, NVIDIA와 network 설정은 직접 관리하고, Kerberos/NFS와 monitoring처럼 별도 모듈이 소유한 기능은 해당 모듈의 점검·복구 명령을 한 순서로 연결한다.

2. 무엇을 확인하고 설정하는가

`new-host-bootstrap` 과 `existing-host-drift` 는 다음 항목을 같은 순서로 사용한다. 따라서 새로운 공통 설정을 추가할 때 두 흐름에 함께 넣을 수 있다.

순서	영역	기존 서버에서 확인하는 것	신규 서버에 설정하는 것
1	기본 접속 조건	inventory 등록, Ansible 접속, 비대화형 sudo, hostname	자동 설정 전 SSH, sudo, hostname, IP와 inventory를 확인
2	공통 OS	Ubuntu 계열 여부, 공통 package 설치 여부	apt repository, NFS, Kerberos와 network 도구 설치

순 서	영역	기존 서버에서 확인하는 것	신규 서버에 설정하는 것
3	Docker Engine	service 활성 상태, daemon 응답, systemd cgroup driver	Docker repository/package와 <code>daemon.json</code> 설정
4	NVIDIA driver	<code>nvidia-smi</code> 가 GPU와 driver version을 정상 출력하는 지, package hold 여부	설정된 driver package 설치와 apt hold
5	NVIDIA Container Toolkit	<code>nvidia-ctk</code> , Docker NVIDIA runtime, containerd NVIDIA runtime	toolkit 설치 후 Docker/containerd runtime 설정
6	Kubernetes node	kubeadm/kubelet/kubectrl, kubelet, cluster join 파일과 node label	Kubernetes package 설치와 kubelet 활성화
7	network tuning	storage NIC 정보, RX queue 4096 이상, 영속화 service	storage NIC RX queue를 4096으로 유지하는 systemd service
8	Kerberos/NFS	realm 설정, machine keytab, service ticket, <code>rpc-gssd</code> , fstab과 mount 상태	client package/config와 GSS 준비 상태 구성
9	monitoring	두 exporter service와 metrics/health endpoint	monitoring 모듈의 exporter 배포 playbook 사용
10	사용자 container 전제조건	사용자 DB 환경, server inventory, Docker 접근	사용자 생성·삭제는 <code>user-lifecycle</code> 을 통해 처리

2.1 NVIDIA version 확인 범위

현재 점검 명령은 `nvidia-smi` 를 실행하여 GPU 이름과 실제 driver version이 출력되는지 확인한다. 따라서 driver가 GPU를 인식하지 못하거나 명령 자체가 실패하는 상태는 찾을 수 있다.

신규 설치 playbook의 기본 package는 현재 `nvidia-driver-580` 이며 설치 후 의도하지 않은 major version 변경을 막기 위해 apt hold한다. 다만 기존 서버의 driver version을 `580` 과 자동 비교하여 불일치 판정을 내리는 규칙은 아직 없다. 현재는 출력된 version을 관리자가 확인해야 한다.

2.2 network 설정 범위

현재 `network-tuning` 이 관리하는 대상은 **스토리지 통신에 사용하는 NIC의 RX queue 크기**다. inventory에서 서버별 storage interface를 읽고, RX queue가 4096 이상인지와 `decs-rx-queue.service` 가 활성화되어 있는지 확인한다.

IP 주소, gateway, DNS와 netplan 전체를 자동으로 설정하는 기능은 아직 없다. 신규 서버의 IP, hostname, SSH와 sudo는 bootstrap을 시작하기 전에 준비해야 하는 조건이다. 향후 모든 서버의 netplan까지 통일하려면 별도의 profile과 검증 규칙을 추가해야 한다.

3. 기존 서버 점검 흐름

기존 FARM/LAB 서버에서는 `existing-host-drift` profile을 사용한다.

공용 inventory에서 서버 선택

- > 공통 기준의 점검 항목 생성
- > 서버별 읽기 전용 명령 확인
- > 차이가 있는 항목의 복구 계획 확인
- > 담당 관리자가 승인한 playbook 실행

전체 서버 또는 특정 서버의 점검 항목을 확인하는 명령은 다음과 같다.

```
cd /home/jy/server_manage

./server-state/bin/server-state check \
--hosts all \
--profile existing-host-drift

./server-state/bin/server-state check \
--hosts farm8 \
--profile existing-host-drift
```

여기서 주의할 점은 현재 **check** 가 실제 서버에 접속하여 결과를 수집하는 명령은 아니라는 것이다. 서버별로 실행할 읽기 전용 Ansible 명령을 **DRY-RUN** 으로 출력한다. 따라서 현재 단계에서는 관리자가 출력된 명령을 실행하거나, 별도 자동화가 그 명령을 실행해야 실제 상태를 알 수 있다.

복구에 사용할 명령도 실행하지 않고 먼저 확인할 수 있다.

```
./server-state/bin/server-state plan \
--hosts farm8 \
--profile existing-host-drift
```

4. 신규 서버 구축 흐름

새 서버를 기존 서버와 같은 상태로 만드는 것이 **new-host-bootstrap** 의 목적이다. 다만 아무것도 설치되지 않은 장비의 전원 투입부터 전부 처리하는 형태는 아니다. 다음 조건은 먼저 준비되어 있어야 한다.

- Ubuntu가 설치되어 있다.
- IP, hostname과 SSH 접속이 준비되어 있다.
- 관리 계정이 비대화형 sudo를 사용할 수 있다.
- 서버가 공용 inventory에 등록되어 있다.

이후 표준 설치 순서를 확인한다.

```
./server-state/bin/server-state plan \  
--hosts farm8 \  
--profile new-host-bootstrap
```

`ansible_playbook/bootstrap_gpu_server.yml`에는 package 설치와 설정 파일 변경을 실제로 수행하는 idempotent task가 들어 있다. 각 단계는 tag로 나뉘며, 먼저 생성된 명령의 host, tag와 변수를 확인한 뒤 `--check --diff`로 예상 변경을 검토한다. 실제 적용은 검토가 끝난 Ansible 명령에서 `--check`를 제거하여 관리자가 실행한다.

다음 항목은 공통 설정이라도 자동으로 밀어 넣지 않고 별도 승인을 요구한다.

- NVIDIA driver 변경: reboot와 실행 중 GPU workload 조정이 필요하다.
- Kubernetes join: cluster별 token과 controller 승인이 필요하다.
- Kerberos machine keytab: FARM과 LAB의 도메인 가입 방식이 다르고 secret을 안전하게 전달해야 한다.
- Kerberos NFS mount: 실행 중 container와 사용자 session에 영향을 줄 수 있다.

Kerberos/NFS의 상세 점검과 복구 순서는 [Kerberos/NFS 매뉴얼](#)에서 관리한다.

5. 현재 구현 수준

이 모듈은 최종적으로 공통 기준과 실제 서버 상태를 자동 비교하고 필요한 복구까지 안전하게 실행하는 것을 목표로 한다. 현재 구현 수준은 다음과 같다.

기능	현재 상태
전체 서버가 따라야 할 공통 profile 정의	구현됨
신규/기존 서버에 같은 profile 순서 사용	구현됨
서버별 점검 명령 생성	구현됨
신규 서버 설정용 Ansible task	구현됨
<code>check</code> 가 원격 서버를 순회하고 결과 판정	아직 구현되지 않음
<code>apply --execute</code> 로 복구 자동 실행	아직 구현되지 않음. 명시적으로 거부됨
주기적인 전체 서버 drift 검사와 알림	아직 구현되지 않음
IP/DNS/netplan 전체 표준화	아직 구현되지 않음

따라서 “모든 서버가 같은 설정인지 확인하고 새 서버를 똑같이 설정한다”는 이해는 맞다. 다만 현재는 **기준, 점검 명령, 복구 playbook을 한곳에 정리한 단계**이고, 한 명령으로 전체 서버를 자동 검사·복구하는 controller까지 완성된 상태는 아니다.

6. 공용 서버 목록

기본 inventory는 `user-lifecycle/server_info/servers.jsonl` 이다. `server-state` 가 별도 서버 목록을 만들지 않는 이유는 IP, SSH port와 논리 server ID를 두 군데에서 관리하여 서로 달라지는 문제를 막기 위해서다.

```
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state list-hosts --hosts farm
./server-state/bin/server-state list-hosts --hosts farm8,lab10
```

selector는 `all`, `farm`, `lab`, 개별 host 이름과 여러 host 조합을 지원한다.

7. profile 구조

`config/profiles.yml` 에는 작은 단위의 profile과 이를 순서대로 묶은 profile set이 있다.

profile set	용도
<code>new-host-bootstrap</code>	신규 SSH-ready 서버의 표준 구축 순서
<code>existing-host-drift</code>	기존 서버의 공통 설정 점검·복구 순서
<code>managed-host</code>	기존 관리 서버용 기본 별칭
<code>monitoring-host</code>	monitoring 항목만 확인할 때 사용

새 공통 설정을 모든 서버에 적용하려면 작은 profile로 추가한 뒤 `new-host-bootstrap` 과 `existing-host-drift` 양쪽에 넣는다. 이렇게 해야 새 서버 설치에는 들어갔지만 기존 서버 점검에서는 빠지거나, 그 반대가 되는 문제를 줄일 수 있다.

각 profile은 다음 정보를 가진다.

- 이 설정을 실제로 소유하는 모듈
- 부작용 없이 상태를 읽는 check
- 차이가 있을 때 사용할 remediation 명령
- 자동 실행할 수 없는 작업의 runbook과 safety level

8. 모듈별 소유권

영역	실제 소유자	<code>server-state</code> 의 역할
공통 OS, Docker, NVIDIA, Kubernetes package, network tuning	<code>server-state</code>	공통 기준과 bootstrap task 관리
NAS, AD, Kerberos와 NFS 정책	<code>kerberos-nfs</code>	점검 순서와 승인된 runbook 연결
exporter와 metrics endpoint	<code>monitoring</code>	배포·점검 playbook 연결

영역	실제 소유자	server-state 의 역할
사용자와 container 생성·삭제	user-lifecycle	공용 inventory 사용과 전제조건 확인
서버 전원과 boot sequence	remote-operations	이 모듈에서 다루지 않음
사용자 container image	container-images	이 모듈에서 image 내부를 변경하지 않음

소유권을 나눈 이유는 `server-state` 에 모든 운영 코드를 복사하지 않고, 실제 담당 모듈의 rollback과 안전 절차를 그대로 사용하기 위해서다.

9. 상태 표시

상태	의미
OK	inventory나 로컬 파일처럼 즉시 확인 가능한 조건이 충족됨
MISSING	필요한 로컬 파일이나 값이 없음
DRY-RUN	실행할 원격 점검 또는 복구 명령만 표시함
MANUAL	도메인 가입, join, mount처럼 담당자 확인이 필요한 작업
UNKNOWN	아직 지원하지 않는 check 또는 remediation 형식

현재 `apply` 도 실제 변경 없이 복구 계획만 출력한다.

```
./server-state/bin/server-state apply \
  --hosts farm8 \
  --profile existing-host-drift
```

`apply --execute` 는 아직 구현되지 않았으며 실행하면 오류로 중단된다.

10. 디렉터리 구조

경로	기능
bin/server-state	CLI 실행 파일
script/cli.py	list-hosts, check, plan, apply 출력
script/inventory.py	공용 JSONL inventory 로드와 host 선택
script/profiles.py	profile set 확장과 서버별 변수 치환

경로	기능
<code>config/profiles.yml</code>	공통 상태, 점검과 복구 명령의 기준
<code>ansible_playbook/bootstrap_gpu_server.yml</code>	신규/기존 서버의 공통 설정 task
<code>ansible_playbook/kerberos_nfs_client_recovery.yml</code>	Kerberos NFS 상태 점검과 제한된 복구
<code>docs/standard-gpu-server-pipeline.md</code>	신규/기존 서버 표준 단계의 개발 문서
<code>tests/</code>	inventory와 profile 처리 unit test

11. 공통 설정 추가 방법

1. 설정을 실제로 관리할 모듈을 정한다.
2. `config/profiles.yml`에 작은 profile을 추가한다.
3. 부작용 없이 읽을 수 있는 항목만 `checks`에 둔다.
4. 복구는 가능하면 Ansible `--check --diff` 명령부터 제공한다.
5. driver, cluster join, keytab과 mount처럼 위험한 작업은 `manual` 또는 `gated`로 둔다.
6. 모든 서버에 필요한 설정이면 `new-host-bootstrap`과 `existing-host-drift`에 모두 추가한다.
7. profile 순서와 서버별 변수 치환 test를 추가한다.

12. 테스트

```
cd /home/jy/server_manage/server-state
python3 -m unittest discover -s tests -v

./bin/server-state --format json check \
  --hosts farm8 \
  --profile existing-host-drift

./bin/server-state plan \
  --hosts lab10 \
  --profile new-host-bootstrap
```

테스트와 dry-run 출력에서는 실제 운영 서버를 변경하지 않는다.

user-lifecycle 설계·운영 매뉴얼

원본: md/system/user-lifecycle.md

역할: 사용자 identity, 그룹, 포트, 컨테이너와 개인 Kerberos/NFS 준비를 하나의 생명주기로 관리한다.

1. 이 모듈이 중심인 이유

DECS 사용자 컨테이너는 Docker object 하나가 아니다. 다음 자원이 서로 같은 identity와 상태를 가져야 하는 분산 작업이다.

- 도메인별 UID MySQL의 사용자, 그룹, 예약 ID, container와 port record
- FARM/LAB target host의 Docker container
- NAS 또는 LAB storage의 사용자 home
- 선택적인 AD RFC2307 user/group과 membership
- host의 root-only user keytab과 ccache refresh timer
- 생성·삭제·연장 안내 메일, DB backup과 Excel export

`user-lifecycle` 는 이 자원의 순서와 실패 처리를 소유한다. 다른 모듈이 DB에 직접 쓰거나 container 생성 규칙을 복제하면 UID, 포트와 실제 Docker 상태가 어긋나므로 공개 `uidctl` CLI를 통과하는 것이 원칙이다.

2. 디렉터리 지도

경로	상태	핵심 기능
<code>script/uid_manager/</code>	primary	Python CLI, model, DB, service 구현
<code>script/uid_manager/services/</code>	primary	create/delete/extend/sync/cleanup/group use case
<code>script/uid_manager/kerberos/</code>	primary	AD, keytab, ccache, NFS 검증 명령과 path 생성
<code>script/ansible_playbook/</code>	primary	storage/NAS home과 host Kerberos identity 준비
<code>script/tests/</code>	primary	fake runner/repository 기반 회귀 테스트
<code>config/*.example.*</code>	template	DB, email, maintenance, Google, admin 설정 구조
<code>config/*.local.*</code>	secret/local	실제 credential과 운영 override; Git 제외
<code>server_info/servers.jsonl</code>	shared authority	FARM/LAB 서버 주소와 SSH/NAT port inventory
<code>nfs_mysql/</code>	data layer	MySQL image, schema와 server config
<code>docker-compose.yml</code>	data layer	local UID DB service/volume

경로	상태	핵심 기능
<code>maintenance/</code>	utility	사용자 삭제, email CSV 갱신 등 관리 작업
<code>ad_backup/</code>	operations	AD backup 생성·검증과 timer 설치
<code>legacy/</code>	compatibility	이전 shell 구현; 신규 기능의 기준이 아님
<code>excel_exports/</code>	generated	사용자 export 결과, source가 아님

`legacy/` 를 즉시 삭제하지 않은 이유는 기존 운영 절차의 비교·복구 근거가 필요하기 때문이다. 그러나 새 비즈니스 규칙은 Python service에 추가하고 테스트 가능하게 유지한다.

3. 코드 구조

CLI와 service 분리

`uid_manager/cli.py` 는 argument parsing, config/repository 생성과 결과 출력만 담는다. 실제 규칙은 service class가 소유한다.

CLI	Service	기능
<code>create-container</code>	<code>ContainerCreateService</code>	identity/port 결정, storage/Kerberos, Docker, DB transaction
<code>delete-container</code>	<code>ContainerDeleteService</code>	대상 단일화, Docker 삭제, DB 비활성화/port 정리
<code>extend-container</code>	<code>ContainerExtendService</code>	만료일 검색·변경과 알림
<code>expired-cleanup</code>	<code>ExpiredCleanupService</code>	만료 대상 계획 또는 실제 정리
<code>sync-containers</code>	<code>ContainerSyncService</code>	DB record와 local Docker 상태 비교·재생성 계획
<code>manage-group</code>	<code>GroupManagementService</code>	AD/DB group과 membership 관리

이 분리는 CLI 외의 timer/test에서도 동일한 규칙을 호출하고, fake repository와 recording runner로 production 접속 없이 계획을 검증하기 위한 것이다.

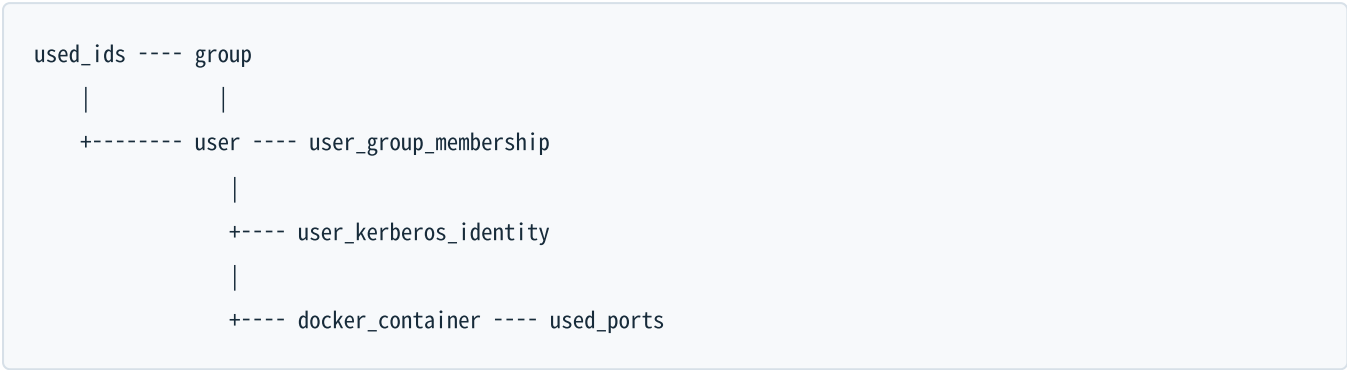
Port와 runner 추상화

`LocalRunner` 와 `AnsibleRunner` 는 subprocess와 원격 shell/playbook 호출을 한 곳에 모은다. `RecordingRunner` 는 실행 대신 command를 기록한다. `PortMapping` 은 host port, container port와 purpose를 함께 보존하여 단순 문자열 port가 SSH/Jupyter/VNC 중 무엇인지 잃지 않게 한다.

`OperationPlan` 은 facts, 단계와 명령을 렌더링한다. `--dry-run` 이 실제 실행 경로와 같은 준비 로직을 통과하므로 문서용 예상 명령과 실제 입력 검증이 달라지는 것을 줄인다.

4. 데이터 모델과 권위

MySQL schema의 주요 관계:



데이터	의미
<code>used_ids</code>	UID/GID 숫자의 전역 예약
<code>group</code>	group 이름과 GID
<code>user</code>	실명, username, UID/GID, 연락처
<code>user_group_membership</code>	supplemental/primary group 관계
<code>user_kerberos_identity</code>	AD realm/SID/RFC2307과 최근 NAS/NFS 관측값
<code>docker_container</code>	image, server, 만료, 생성자와 활성 상태
<code>used_ports</code>	host port, 용도와 container record 연결

foreign key와 unique index를 쓰는 이유는 application 검증만으로 막기 어려운 중복 UID/GID, container와 port 충돌을 DB에서도 거부하기 위해서다. FARM과 LAB은 DB endpoint가 다를 수 있으므로 `AppConfig`가 domain별 host를 선택한다.

5. Container 생성 흐름

5.1 준비와 identity 채택

`ContainerCreateService.prepare()`는 이름, domain/server, 날짜, image와 port를 검증한 뒤 UID/GID source를 결정한다. 신규 사용자의 UID 우선순위는 다음과 같다.

1. DB에 기존 사용자가 있으면 DB UID/GID
2. 운영자가 명시한 `--uid / --gid`
3. Kerberos 모드에서 기존 AD RFC2307 identity
4. 기존 storage home의 숫자 owner
5. 모두 없으면 DB의 다음 사용 가능한 ID

기존 자원을 무조건 새 ID로 덮지 않고 채택하는 이유는 이미 소유권이 있는 NFS home이나 AD object를 고아로 만들지 않기 위해서다. 여러 source가 서로 다른 값을 말하면 자동 보정하지 않고 validation error로 중단한다.

port는 DB 예약과 실제 target host Docker publish port를 함께 확인한다. 기본 SSH/Jupyter, 선택적 VNC와 추가 container port를 배정하거나 `--fixed-port-mappings` 로 host:container[:purpose]를 명시할 수 있다.

5.2 실행 순서

입력 검증과 OperationPlan

|

v

target image inspect/pull

|

v

storage home + optional AD/keytab/ccache 준비

|

v

NSS/idmap/실제 NFS write 검증

|

v

docker run + inspect/port 검증

|

v

DB user/group/identity/container/port 단일 transaction

|

v

email + DB backup + export

storage와 Kerberos write 검증을 Docker/DB 확정 전에 두는 이유는 사용자가 로그인했지만 home에 쓸 수 없는 반쪽 container를 만들지 않기 위해서다. Docker 생성 뒤 DB write는 한 transaction으로 묶는다. DB 단계가 실패하면 transaction을 rollback하고 방금 만든 container 정리를 시도하여 실제 상태와 record가 갈라지는 시간을 줄인다.

`--no-db-record` 는 특수 검증용으로 Docker를 만들되 DB user/group/container/port write를 생략한다. 일반 운영 생성에는 사용하지 않는다. DB에 없는 container는 GPU user attribution, 만료 정리와 sync에서 정상 관리 대상이 아니기 때문이다.

5.3 domain별 storage

일반 LAB root_squash 경로는 storage server에서 실제 export home을 만들고 선택한 UID/GID로 owner를 설정한 뒤, target host에 mount된 `/home/tako<N>/share/user-share` 를 container `/home` 에 bind한다.

FARM은 NAS의 user-share 구조를 사용한다. Kerberos가 아닌 모드도 target host root가 NFS root_squash 때문에 직접 owner를 바꾸려 하지 않고 storage/NAS 소유 playbook을 통해 준비한다.

6. Kerberos 활성화 흐름

6.1 공통 보안 모델

```
AD user/private group + RFC2307
  |
  v
root-only keytab on target host
  |
  v
systemd refresh -> /run/user/<uid>/krb5cc
  |
  v
container receives ccache only
```

사용자 password를 DB나 container에 저장하지 않는다. AD에서 keytab을 export한 뒤 target host의 `/etc/decs-krb/keytabs/`에 mode `0400`으로 설치한다. refresh service는 ticket을 renew하거나 만료 여유가 부족하면 keytab으로 재발급하고, container에는 ccache만 전달한다.

keytab rotation은 `--rotate-kerberos-keytab`처럼 명시적으로 요청한다. 정상 container 재생성이나 만료 연장이 credential rotation을 뜻하지 않게 하기 위해서다.

6.2 FARM

FARM 흐름은 AD user/group과 RFC2307을 보장하고 NAS mapping과 home을 준비한 뒤 target host에서 해당 UID와 ccache로 실제 NFS write를 수행한다.

공유 NAS GSS/idmap service restart는 기본적으로 꺼져 있다. 새 identity의 cache 문제가 의심되더라도 NAS service 재시작은 모든 client의 GSS context를 끊을 수 있으므로 명시적 유지보수 옵션으로만 허용한다.

6.3 LAB

LAB Kerberos PoC/지원 흐름은 LAB Samba AD, storage의 전용 Kerberos export, target host의 NSS/idmap과 NFSv4.1을 함께 확인한다. storage와 client의 NFSv4 idmap domain이 다르면 owner가 `nobody`로 보이거나 `chgrp`가 실패할 수 있다. 따라서 ticket 성공만으로 완료하지 않고 숫자 owner와 실제 write를 검증한다.

6.4 DB에 남기는 Kerberos 정보

DB에는 password/keytab이 아니라 다음 비밀이 아닌 identity metadata만 둔다.

- AD username, realm, NetBIOS domain
- domain/object SID
- AD `uidNumber`, `gidNumber`
- 최근 NAS internal UID/GID
- 최근 NFS에서 본 UID/GID와 검증 시각

SID와 최근 관측값을 보존하면 같은 username이 재생성되었거나 mapping이 달라진 상황을 단순 문자열 일치보다 안전하게 발견할 수 있다.

7. 주요 CLI 사용법

설치/실행 위치:

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m pip install -e .
uidctl --help
```

생성 dry-run

```
uidctl create-container \
  --name '홍길동' \
  --username hong \
  --server-id FARM8 \
  --expiration-date 2026-12-31 \
  --image decs \
  --version cuda12.5-tf2.20-ubuntu22.04-260706 \
  --created-by admin \
  --email hong@example.org \
  --phone 000-0000-0000 \
  --dry-run
```

Kerberos/VNC는 필요할 때 명시한다.

```
uidctl create-container ... --enable-kerberos --enable-vnc --dry-run
```

삭제

DB record 검색 조건으로 대상이 하나인지 먼저 확인하고 dry-run한다.

```
uidctl delete-container \
  --server-id FARM8 \
  --container-name hong \
  --dry-run
```

만료 연장과 정리

연장은 기본적으로 계획만 만들고 실제 변경에 `--apply`가 필요하다.

```
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30  
  
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30 \  
  --apply
```

만료 정리도 대상 날짜와 domain을 제한해 먼저 dry-run한다.

```
uidctl expired-cleanup \  
  --today 2026-07-17 \  
  --domains FARM,LAB \  
  --dry-run
```

sync와 group

```
uidctl sync-containers --domain FARM --dry-run  
uidctl manage-group show --domain FARM --group research  
uidctl manage-group add-user \  
  --domain FARM \  
  --group research \  
  --user hong \  
  --dry-run
```

`--force`, `--auto-delete`, 실제 group delete는 영향 범위를 확인한 뒤 사용한다.

8. 설정과 secret

공개 template:

- `config/db_config.example.env`
- `config/email_config.example.env`
- `config/daily_maintenance.example.env`
- `config/google-client.example.json`
- `config/reminder_admins.example.txt`
- `ad_backup/config.example.env`

실제 값은 대응하는 `.local.*` 또는 ignored 운영 파일에 둔다. 문서, log, OperationPlan과 exception에 DB password, SMTP credential, Google secret, AD password, keytab 내용이 나타나지 않도록 command redaction을 유지한다.

`config/network_topology.json` 과 `server_info/servers.jsonl` 은 secret 저장소가 아니다. 접근 경로에 필요한 주소와 port만 넣고 password/private key는 외부 SSH/Ansible 설정이 관리한다.

9. Post action과 생성물

성공한 생명주기 작업 뒤 `PostActions` 가 이메일, DB backup과 export 도구를 호출한다. 핵심 transaction과 알림을 분리한 이유는 메일 실패가 identity/DB 일관성을 되돌리는 근거가 되지 않기 때문이다. 반대로 DB transaction이 실패했는데 성공 메일을 보내지 않도록 실행 순서는 transaction 뒤에 둔다.

`excel_exports/`, `AD_backups/`, `server_info` 의 생성된 raw 정보는 운영 artifact다. source code처럼 수정하거나 매뉴얼의 권위 있는 입력으로 삼지 않는다. 보존·암호화·삭제 주기는 별도 운영 정책에 따라야 한다.

10. 테스트

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m unittest discover -s tests -v
```

또는 프로젝트 test runner:

```
python3 tests/run_tests.py
```

중요 회귀 범위:

- 자동/고정 port 할당과 중복 거부
- DB/AD/storage 기존 identity 채택 우선순위
- create 실패 시 DB rollback과 container cleanup
- dry-run에서 원격/DB 변경이 없는지
- FARM/LAB Kerberos command와 path 차이
- group 삭제/primary 변경의 안전 gate
- sync가 DB에 없는/멈춘 container를 구분하는지
- command/log에서 password redaction

shell legacy test는 호환성 확인용이며 Python service test를 대체하지 않는다.

11. 변경 가이드

새 lifecycle command

1. request/result 또는 record를 `models.py`에 정의한다.
2. 비즈니스 순서를 `services/`의 class로 구현한다.
3. DB query는 `Repository` protocol과 `MySqlRepository`에 추가한다.
4. 외부 명령은 runner를 통하고 secret redaction을 적용한다.
5. CLI에는 parsing과 dependency 조립만 추가한다.
6. fake repository/runner를 사용한 dry-run, success와 partial failure test를 작성한다.

schema 변경

- 기존 운영 DB에 idempotent하게 적용되는 migration/ensure path를 준비한다.
- unique/foreign key와 기존 데이터 backfill 순서를 검토한다.
- FARM/LAB DB를 독립적으로 적용·검증한다.
- exporter가 읽는 column과 query 호환성을 `monitoring`에서 함께 확인한다.

Kerberos 변경

- password/keytab이 DB, plan과 log에 들어가지 않는지 확인한다.
- DB, AD, NAS/NFS와 container identity 불변 조건을 test한다.
- 공유 NAS service restart를 정상 경로에 추가하지 않는다.
- 운영 policy 변경이면 `kerberos-nfs/docs/farm.md` 또는 `docs/lab.md`를 갱신한다.

12. 운영 안전 수칙

- 모든 destructive 명령은 domain, server와 대상 container가 하나인지 확인한다.
- 실제 생성 전 `--dry-run`의 identity source, mount, image와 port를 검토한다.
- 기존 AD 또는 storage identity와 요청 UID/GID 충돌을 강제로 덮지 않는다.
- DB와 실제 Docker 상태가 어긋나면 원인을 확인하고 `sync --auto-delete`를 바로 사용하지 않는다.
- keytab과 사용자 password를 container나 backup export에 포함하지 않는다.
- `legacy/`를 수정해 신규 규칙을 우회하지 않는다.
- DB 없는 test container는 이름, 수명과 정리 책임을 명확히 한다.